

# Tarea Contenedores TSIC2

---

## Introducción

---

se realizan las tareas propuestas en el artefacto copartido [artifact](#)

## Parte 1: solo el servidor web con python

---

se Crearon lo archivos indicados **Containerfile** y **server.py** en la carpteta web-mython

```
└─ web-python
   └─ Containerfile
      └─ server.py
```

además de los scripts:

- **01-01-run-web-python.sh** Que crea el contenedor basad en el Containerfile y el arhivo server.py
- **01-02-verificar-respuesta.sh** Que pueba el resultado realizando dos consultas

Ejecución de **01-01-run-web-python.sh**

```

martin@rocky10:~/tsic-tarea-contenedores$ sh 01-01-run-web-python.sh
Se crea el servidor web basado en server.py y Containerfile
STEP 1/8: FROM registry.access.redhat.com/ubi9/python-311
STEP 2/8: LABEL description="Webserver en Python puro con http.server"
--> Using cache 812eb6426372f3a6e1b20daee018c2d5c9e907eb563d010f0d760f86befe2e5b
--> 812eb6426372
STEP 3/8: WORKDIR /opt/app-root/src
--> Using cache d18a2a29d8f4193f18ef485fa0c998e8182e7c00d1bd13c94275b0f90431da3a
--> d18a2a29d8f4
STEP 4/8: COPY --chown=1001:0 server.py .
--> Using cache af44ac444d471bc1230787213b56d99dd79f148bb15cd486abf09bb9442a7a55
--> af44ac444d47
STEP 5/8: ENV PORT=8080      PYTHONUNBUFFERED=1
--> Using cache 1fcb09d5d5cecf31f3a314de6299f55d020f470b67a2c594d798a07041dd9eb6
--> 1fcb09d5d5ce
STEP 6/8: EXPOSE 8080
--> Using cache 461db561ff7857deee975f319c3ac561f88989d374feff4d043bb667579ba337
--> 461db561ff78
STEP 7/8: USER 1001
--> Using cache 8774f7b2a20739fa19d011148f1dd7981b10e50d887c5e227e27ee0a37fa8036
--> 8774f7b2a207
STEP 8/8: CMD ["python", "server.py"]
--> Using cache 526556678c6456e57f90658b592279d20a6e7c76930ec6b8177e61970a44e112
COMMIT py-web:1.0
--> 526556678c64
Successfully tagged localhost/py-web:1.0
526556678c6456e57f90658b592279d20a6e7c76930ec6b8177e61970a44e112
a952559b3b3219888baee91d695892f4bc540c56a2a3ada410fd3f1d149f3e77

```

Ahora se listan los contenedores

```

martin@rocky10:~/tsic-tarea-contenedores$ podman container ls
CONTAINER ID   IMAGE                                COMMAND                                CREATED        STATUS
PORTS          NAMES
f493aa51795b   localhost/pg-usuarios:1.0           postgres                                49 minutes ago Up 49 minutes
0.0.0.0:5432->5432/tcp   db
a952559b3b32   localhost/py-web:1.0                python server.py                       44 seconds ago Up 44 seconds
0.0.0.0:8080->8080/tcp   web

```

ahí se encuentra el contenedor **web**

ahora se prueba que el contenedor responda ejecutando **01-02-verificar-respuesta.sh**

```
martin@rocky10:~/tsic-tarea-contenedores$ sh 01-02-verificar-respuesta.sh
Verificar que el servidor responda
curl a: http://127.0.0.1:8080
<h1>Hola desde Podman</h1><p>Contenedor: a952559b3b32</p><p>Hora: 2026-09-25 00:43:34</p>
curl a: http://127.0.0.1:8080/health
<h1>Hola desde Podman</h1><p>Contenedor: a952559b3b32</p><p>Hora: 2026-09-25 00:43:34</p>
```

Para que la prueba fuera exitosa se tuvo que cambiar la URL a la que se hace la solicitud por una URL basada en la ip **loopbak** en lugar de usar localhost, ya que así no funcionaba

Se detiene y elimina el contenedor para hacer mas pruebas

```
martin@rocky10:~/tsic-tarea-contenedores$ podman container stop web
WARN[0010] StopSignal SIGTERM failed to stop container web in 10 seconds, resorting to
SIGKILL
web
martin@rocky10:~/tsic-tarea-contenedores$ podman container rm web
web
```

## Parte 2: postgres con usuarios

En la carpeta db\_postgres se crea en **Containerfile** y el archivo de inicialización **init.sql**

```
db-postgres/
├─ Containerfile
└─ init.sql
```

tambien se crean los scripts

- **02-01-crea-db-posgres.sh** : Para crear el contenedor de postgres
- **02-02-usa-posgres.sh** : para hacer las pruebas

Se ejecuta el primero **02-01-crea-db-posgres.sh**

```

martin@rocky10:~/tsic-tarea-contenedores$ sh 02-01-crea-db-posgres.sh
A CONTINUACIÓN DE CREARÁ EL CONTENDOR POSTGRES
STEP 1/5: FROM docker.io/library/postgres:16
STEP 2/5: LABEL description="PostgreSQL con usuarios y datos iniciales"
--> Using cache 0f9ec7146c025013c58ca0240a1a165df02fec2cf55075df6e48e327afb51e8a
--> 0f9ec7146c02
STEP 3/5: COPY init.sql /docker-entrypoint-initdb.d/01-init.sql
--> Using cache a6794f9acle08daa9372becff36b9ac3e468b17ec976476eb7c8806af70699e7
--> a6794f9acle0
STEP 4/5: ENV TZ=America/Mexico_City
--> Using cache 5d6aalf9a463575873a20af3ba8323466820507062792e40a6b45f3640117bc6
--> 5d6aalf9a463
STEP 5/5: EXPOSE 5432
--> Using cache 3139517553f1190bcb1d38bbcac876d69c90dfb04dfe0354c1bdbbdf9421b30f
COMMIT pg-usuarios:1.0
--> 3139517553f1
Successfully tagged localhost/pg-usuarios:1.0
3139517553f1190bcb1d38bbcac876d69c90dfb04dfe0354c1bdbbdf9421b30f
Error: volume with name pgdata already exists: volume already exists
f493aa51795b53254e17898086d40960b3d3ef5340685075eeb10aefb9483463

```

Se listan los contenedores

```

martin@rocky10:~/tsic-tarea-contenedores$ podman container list

```

| CONTAINER ID           | IMAGE                     | NAMES | COMMAND          | CREATED        | STATUS        |
|------------------------|---------------------------|-------|------------------|----------------|---------------|
| ecc5b1b29151           | localhost/py-web:1.0      |       | python server.py | 28 minutes ago | Up 28 minutes |
| 0.0.0.0:8080->8080/tcp |                           | web   |                  |                |               |
| f493aa51795b           | localhost/pg-usuarios:1.0 |       | postgres         | 21 seconds ago | Up 21 seconds |
| 0.0.0.0:5432->5432/tcp |                           | db    |                  |                |               |

y se ejecuta el segundo script **02-02-usa-posgres.sh** para hacer las pruebas

```

martin@rocky10:~/tsic-tarea-contenedores$ sh 02-02-usa-posgres.sh
MOSTRAR LOS USUARIOS CREDOS

                                List of roles
Role name |                               Attributes
-----+-----
app_user  |
postgres | Superuser, Create role, Create DB, Replication, Bypass RLS
reporte   |

CONSULTAR COMO APP_USER Y COMO REPORTE
id | nombre | cantidad | creado
---+-----+-----+-----
1 | Servidor ProLiant | 4 | 2026-09-24 17:35:15.314784
2 | Switch 48p | 2 | 2026-09-24 17:35:15.314784
3 | Disco NVMe 3.84TB | 12 | 2026-09-24 17:35:15.314784
(3 rows)

count
-----
3
(1 row)

REPORTE NO PUEDE ESCRIBIR, DEBE FALLAR
ERROR: permission denied for table productos

```

Las cuales son correctas

Se detiene y elimina el contenedor para hacer mas pruebas

```

martin@rocky10:~/tsic-tarea-contenedores$ podman container stop db
db
martin@rocky10:~/tsic-tarea-contenedores$ podman container rm db
db

```

## Parte 3: Actualizar y crear usuarios desde un archivo en ubuntu

Se crea la carpeta **ubuntu-usuarios**

dentro de esta los archivos Containerfile y usuarios.txt

```

.
├── Containerfile
└── usuarios.txt

```

se ejecuta el primer script **03-01-containerfile-init-update.sh**

```
martin@rocky10:~/tsic-tarea-contenedores$ sh 03-01-containerfile-init-update.sh
El containerfile: Creara , acutualizara y agregara los usuarios a ubutu
STEP 1/8: FROM docker.io/library/ubuntu:24.04
Trying to pull docker.io/library/ubuntu:24.04...
Getting image source signatures
Copying blob eddled89f0d4 done    |
Copying config 6232b38791 done    |
Writing manifest to image destination
STEP 2/8: LABEL description="Ubuntu actualizado con usuarios desde usuarios.txt"
--> c04ae0f415ae
STEP
[...]

[...]
COMMIT ubuntu-usuarios:1.0
--> 0b09ede32789
Successfully tagged localhost/ubuntu-usuarios:1.0
0b09ede3278963e5253413a4e887ab7359e78af4f43de10607d20ef78eb25a04
```

se ejecuta el seundo script

```
martin@rocky10:~/tsic-tarea-contenedores$ sh 03-02-verifica-usuarios.sh
Verifica que los usuarios si estén creados
devops:x:1001:daniel,ana,alumno1,alumno2
alumno1
alumno2
ana
daniel
ubuntu
root@4a79c77b7772:/home# ls
alumno1 alumno2 ana daniel ubuntu
root@4a79c77b7772:/home#
```

Donde efectivamente se han creado los usuarios del archivo de texto

Se listan los contenedores

```
martin@rocky10:~/tsic-tarea-contenedores$ podman container ls -a
```

| CONTAINER ID             | IMAGE                          | COMMAND              | CREATED            |      |
|--------------------------|--------------------------------|----------------------|--------------------|------|
| STATUS                   | PORTS                          | NAMES                |                    |      |
| 932501104aae             | docker.io/library/nginx:latest | nginx -g daemon o... | 6 days ago         |      |
| Exited (0) 292 years ago | 80/tcp                         | nginx01              |                    |      |
| a3b4217fa43d             | docker.io/library/nginx:latest | nginx -g daemon o... | 6 days ago         |      |
| Exited (0) 5 days ago    | 0.0.0.0:8081->80/tcp           | nginx03              |                    |      |
| a041d76b26b3             | docker.io/library/nginx:latest | nginx -g daemon o... | 6 days ago         |      |
| Exited (0) 292 years ago | 0.0.0.0:8082->80/tcp           | nginx04              |                    |      |
| ecc5b1b29151             | localhost/py-web:1.0           | python server.py     | 59 minutes ago     | Up   |
| 59 minutes               | 0.0.0.0:8080->8080/tcp         | web                  |                    |      |
| f493aa51795b             | localhost/pg-usuarios:1.0      | postgres             | 31 minutes ago     | Up   |
| 31 minutes               | 0.0.0.0:5432->5432/tcp         | db                   |                    |      |
| 4a79c77b7772             | localhost/ubuntu-usuarios:1.0  | /bin/bash            | About a minute ago | Up 2 |
| minutes                  |                                | ubu                  |                    |      |

donde efectivamente aparece el conrenedor ubu, sin embargo este desaparece al momento cerrar la ventana de la terinal del cotnenodor ubuntu

## Eliminación del contenedor

Este no se encuentra en la lista de contenedores ni activos ni detenidos

```
martin@rocky10:~/tsic-tarea-contenedores$ podman container list --all
```

| CONTAINER ID  | IMAGE                          | COMMAND              | CREATED    | STATUS       |
|---------------|--------------------------------|----------------------|------------|--------------|
| PORTS         | NAMES                          |                      |            |              |
| 932501104aae  | docker.io/library/nginx:latest | nginx -g daemon o... | 6 days ago | Exited (0)   |
| 292 years ago | 80/tcp                         | nginx01              |            |              |
| a3b4217fa43d  | docker.io/library/nginx:latest | nginx -g daemon o... | 6 days ago | Exited (0) 5 |
| days ago      | 0.0.0.0:8081->80/tcp           | nginx03              |            |              |
| a041d76b26b3  | docker.io/library/nginx:latest | nginx -g daemon o... | 6 days ago | Exited (0)   |
| 292 years ago | 0.0.0.0:8082->80/tcp           | nginx04              |            |              |

ubuntu se encuentra en las imágenes

```

martin@rocky10:~/tsic-tarea-contenedores$ podman image list
REPOSITORY                                TAG                IMAGE ID           CREATED
SIZE
localhost/ubuntu-usuarios                1.0                0b09ede32789      34 minutes ago
98.6 MB
localhost/pg-usuarios                    1.0                3139517553f1      About an hour ago  458
MB
localhost/py-web                          1.0                526556678c64      2 hours ago        1.1
GB
docker.io/library/postgres                16                 1b3c642526f8      6 days ago         459
MB
localhost/nginx                          latest             878c33739a8a      9 days ago         174
MB
docker.io/library/nginx                  latest             878c33739a8a      9 days ago         174
MB
docker.io/library/ubuntu                 24.04              6232b3879100      13 days ago
80.7 MB
registry.access.redhat.com/ubi9/python-311 latest             c04750c4c98c      3 months ago       1.1
GB

```

## Conclusiones

Con esta tarea tuvimos la oportunidad de aprender mas sobre como se crean, destruyen y manejan contenedores de podman. Se nota que la instalar, modificar y eliminar aplicaciones en contenedores es mucho más limpio y ordenado.